

DZMarket - Architecture et Conception UML

Date: 18/02/2026

Portee: application Flutter (Android/Web), backend Supabase self-host, infra Hetzner.

1. Objectif d'architecture

- Supporter un marketplace C2C/C2B avec chat, offres, commandes, logistique et notifications.
- Garder une architecture simple a exploiter (petite equipe), mais extensible.
- Isoler clairement:
- `Presentation` (UI Flutter)
- `Application/Domain` (regles metier dans services)
- `Data/Infrastructure` (Supabase, Storage, Edge Functions, SMTP)

2. Vue Contexte (UML - niveau systeme)

Acteurs principaux:

- Acheteur
- Vendeur
- Admin
- Service logistique (Yalidine, ZR Express, Ecotrack, Guepex)
- Service email SMTP (Brevo)

Systeme:

- DZMarket App (Flutter)
- DZMarket API (Supabase/Kong)
- DZMarket Data (PostgreSQL + Storage S3/MinIO)

Interactions:

- Acheteur/Vendeur -> DZMarket App: navigation, chat, offres, achat, suivi.
- DZMarket App -> DZMarket API: Auth, CRUD metier, realtime, fonctions Edge.
- DZMarket API -> DZMarket Data: lecture/ecriture des tables metier.
- DZMarket API -> SMTP: envoi mails confirmation/reset.
- Edge Functions -> APIs transporteurs: validation token, estimation, creation expéditions.

3. Vue Containers (UML - niveau deployment logique)

Containers applicatifs:

- Flutter App (Android/Web)
- Caddy (TLS + reverse proxy)
- Kong (API gateway Supabase)
- GoTrue (Auth)
- PostgREST (REST DB)
- Realtime (events/chat)
- Storage API

- Edge Runtime (fonctions)
- PostgreSQL
- MinIO (objets S3)

Topologie cible Hetzner:

- `dzm-app-01`: Caddy + stack Supabase self-host.
- `dzm-db-01`: PostgreSQL (selon runbook cible 3 serveurs).
- `dzm-storage-01`: MinIO objets.

Flux reseau principal:

- 1 Client -> `https://app.dzmarket.pro` (frontend web)
- 2 Client -> `https://api.dzmarket.pro` (API Auth/REST/Storage)
- 3 API interne -> PostgreSQL + MinIO + Edge Functions
- 4 Auth -> SMTP Brevo

4. Architecture interne Flutter (UML - composants)

Organisation code:

- `lib/src/features/\*`: ecrans et logique UI par domaine (auth, chat, listings, orders, profile, etc.)
- `lib/src/services/\*`: services applicatifs/metier (auth_service, order_service, shipping_service, etc.)
- `lib/src/models/\*`: objets metier (Product, Order, Offer, Profile, Shipment, Message, ...)
- `lib/src/router.dart`: routage et gardes auth.

Composants cle:

- `DZMarketApp` (bootstrap, theme, locale, router)
- `GoRouter` (redirections auth + callbacks email)
- `AuthService` (signup/login/reset/resend + mapping erreurs)
- `NotificationService` (local notifications, badges)
- `ShippingService` + fonctions Edge (orchestration livraison)

5. UML Use Cases (fonctionnel)

Use cases Acheteur:

- S'inscrire / se connecter / recuperer mot de passe
- Parcourir annonces
- Envoyer une offre
- Passer commande
- Suivre commande et discuter avec vendeur

Use cases Vendeur:

- Publier annonce
- Accepter/refuser offres
- Generer expedition/label
- Suivre retours et messagerie

Use cases Admin:

- Moderer contenu
- Superviser operations et erreurs

6. UML Classes (modele domaine simplifie)

Classes principales:

- `Profile` (id, email, full_name, phone, role, status, lang, location)
- `Product` (id, seller_id, title, price, stock, negotiable, category, status)
- `Offer` (id, product_id, buyer_id, amount, status, created_at)
- `Order` (id, product_id, buyer_id, seller_id, amount, status, delivery_mode)
- `Shipment` (id, order_id, provider, tracking_number, label_url, status)
- `Conversation` (id, product_id/order_id, participants, unread counters)
- `Message` (id, conversation_id, sender_id, body, type, created_at)

Relations metier:

- Un `Profile` vendeur possede N `Product`.
- Un `Product` recoit N `Offer`.
- Une `Offer` acceptee peut produire une `Order`.
- Une `Order` peut avoir 0..1 `Shipment`.
- Une `Conversation` contient N `Message`.

7. UML Sequence - Signup avec confirmation email

Sequence:

- 1 Utilisateur saisit formulaire signup (email, mdp, profil).
- 2 `SignUpPage` appelle `AuthService.signup(...)`.
- 3 `AuthService` appelle Supabase Auth (`signup`) avec `emailRedirectTo`.
- 4 GoTrue cree utilisateur et envoie email via SMTP Brevo.
- 5 Utilisateur clique lien email.
- 6 Route `/auth/callback` parse le token/code.
- 7 `AuthCallbackPage` valide la session/token via Supabase.
- 8 Redirection vers `/sign-in?confirmed=1`.

8. UML Sequence - Reset mot de passe

Sequence:

- 1 Utilisateur demande reset depuis `ResetPasswordPage`.
- 2 `AuthService.sendPasswordResetEmail` envoie la demande.
- 3 GoTrue envoie email recovery (template branding DZMarket).
- 4 Utilisateur clique lien recovery.
- 5 `/auth/callback` etablit session temporaire recovery.
- 6 Redirection vers `/reset-password`.
- 7 Utilisateur saisit nouveau mot de passe + confirmation.
- 8 `auth.updateUser(password: ...)` applique le changement.

9. UML Sequence - Offre vers commande/livraison

Sequence:

- 1 Acheteur envoie offre sur produit negociable.
- 2 Vendeur accepte/refuse.
- 3 Si acceptee: creation commande avec prix verrouille a l'offre acceptee.
- 4 Creation shipment via Edge Function (`create\_shipment`) selon transporteur vendeur.
- 5 Label stocke dans Storage (bucket `labels`).
- 6 Etat commande + chat systeme mis a jour.

10. Decisions techniques structurantes

- Flutter unique codebase pour Android + Web.
- Supabase self-host pour controle infra/cout/donnees.
- Caddy pour TLS automatique et reverse proxy.
- Edge Functions pour integrer transporteurs et logique externe.
- Templates email heberges sous `app.dzmarket.pro/auth-email/\*`.
- I18n FR/AR avec fallback local + surcharge service de traduction.

11. Qualites et points de vigilance

Points forts:

- Separation claire UI / services / data.
- Forte capacite d'iteration produit.
- Pipeline deploy simple (build web + zip + deploy Caddy/Supabase).

Points a surveiller:

- Gestion des secrets (rotation SMTP/API keys).
- Observabilite unifiee (errors auth, delivery failures, latency).
- Tests E2E de non-regression auth/chat/commande avant chaque release.

12. Roadmap architecture recommandee

- 1 Standardiser les contrats API internes (DTO + validation stricte).
- 2 Ajouter diagrammes UML maintenus versionnes par release.
- 3 Renforcer CI: tests unitaires + integration + smoke prod.
- 4 Introduire tracing technique (request-id de bout en bout).
- 5 Preparer une strategie blue/green pour deploy sans interruption.

13. Annexes - fichiers de reference projet

- `lib/src/router.dart`
- `lib/src/services/auth\_service.dart`
- `lib/src/features/auth/\*`
- `lib/src/services/shipping\_service.dart`
- `supabase/functions/\*`

- ``infra/HETZNER_MIGRATION_RUNBOOK.md``
- ``infra/hetzner/docker-compose.auth-mail-templates.yml``